Save It All: Enabling Full Parameter Tuning for Federated Large Language Models via Bicycle Black Gradient Descent

Anonymous ACL submission

Abstract

The advent of large language models (LLMs) has revolutionized the deep learning paradigm, yielding impressive results across a wide array of tasks. However, the pre-training or fine-tuning of LLMs within a federated learning (FL) framework poses substantial challenges, including considerable computational and memory resource demands, as well as communication bottlenecks between servers and clients. Existing solutions either make the unrealistic assumption that the entire model is exchanged for training, or apply parameter-effective fine-tuning methods from centralized learning to train LLMs in FL which tend to underperform during training or fine-tuning stages due to the limited search subspace of parameter updating. In this paper, we introduce a novel method for the efficient training and fine-tuning of LLMs in FL, with minimal resource consumption. Our approach, termed FedCyBGD, utilizes Cycle Block Gradient Descent to periodically update the model. In particular, we design a compression scheme for FedCyBGD, aiming to further decrease the model download cost. It enables full parameter training in FL with only selected block updates and uploads, thereby reducing communication, computation, and memory costs. Our method achieves state-of-the-art performance for FL LLM training, while significantly reducing associated costs.

1 Introduction

Large language models (LLMs) have demonstrated exceptional performance across a multitude of domains, including language understanding (Brown et al., 2020; Devlin et al., 2018; Achiam et al., 2023; Meta, 2024; Yang et al., 2024), computer vision (Radford et al., 2021; Fang et al., 2023), reasoning (Zhuang et al., 2023a), and speech recognition (Radford et al., 2023). By pre-training on extensive datasets, these models can learn general representations that prove beneficial for a wide array of downstream tasks (Xiao et al., 2023a; Zhuang et al., 2023b).

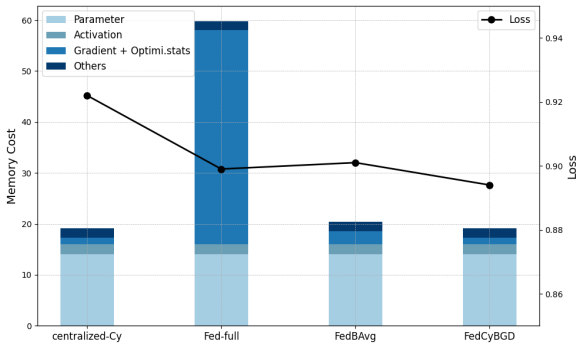


Figure 1: **Observation on Federated Learning.** Bar graphs represent the estimated memory usage for full parameter tuning of an LLaMA-7B model on a single device and the line graph represents the loss across different training paradigms. 'Centralized-Cy' denotes centralized training with cyclical block updates, 'Fed-full' refers to federated full parameter tuning with complete model communication to clients, 'FedBAvg' signifies federated training with block updates where the server selects clients for tuning and aggregates updates, and 'FedCyBGD' represents our approach, where clients cyclically participate in block tuning.

Despite the significant impact of LLMs in artificial intelligence and other fields, their deployment on edge devices, such as smartphones and IoT devices, is hindered by high storage and processing resource requirements. Furthermore, the growing concern over data privacy issues (Yang et al., 2020; Li et al., 2021; Mothukuri et al., 2021) has led to an increased interest in federated learning (FL) (McMahan et al., 2017; Kairouz et al., 2019), indicating a rising demand for edge-based training. FL is a distributed learning paradigm that enables edge devices to collaboratively train models without sharing raw data (McMahan et al., 2017). This approach addresses privacy and communication concerns and has proven valuable in sectors such as healthcare (Kaissis et al., 2020), en-

ergy (Saputra et al., 2019), and manufacturing (Qu et al., 2020). Given its data silo nature, federated learning has emerged as an effective solution for privacy-preserving LLM training (Fan et al., 2023; Kuang et al., 2023; Ye et al., 2024). Traditional FL assumes that the client possesses sufficient computational power, typically in the form of GPUs, to train computation-intensive models (Wang et al., 2021). However, with the increasing size of model parameters and datasets, this assumption becomes increasingly impractical to fulfill in real-world scenarios (Tao et al., 2022; Che et al., 2023).

Notably, the pre-training or fine-tuning of LLMs incurs substantial memory and computational costs that are prohibitive for edge devices (Zhao et al., 2024), and communication costs remain a primary bottleneck when applied in FL (Wu et al., 2024). Some works have employed parameter-effective-fine-tuning (PEFT) for fine-tuning LLMs in FL, such as Lora, which fine-tunes the LLM based on local datasets. The principle of PEFT is to represent parameter updates in a much lower-dimensional subspace, which can potentially limit downstream performance (Zhang et al., 2024). We will discuss the advantages of our framework over PEFT methods in detail in the next section and show that most existing PEFT methods are compatible with our framework.

Consequently, the challenges of training LLMs for FL and the objective of our paper can be summarized as follows:

Reducing computation, memory, and communication costs; Enabling effective federated large language model training.

To address these challenges and achieve this goal, we propose a simple yet effective training paradigm called **FedCyBGD**, which trains LLMs in a **Fed**erated Learning environment using a **Cy**clic **B**lock **G**radient **D**escent approach. In terms of memory, computation, and communication costs, **FedCyBGD** aims to ***save it all***. It employs an alternating minimization strategy, training *responsible blocks* on edge devices and periodically syncing their updates to the server. For *unresponsible blocks*, devices download a surrogate network, a scaled-down model with similar functionality but fewer parameters, negating the need for additional updates or uploads. This method not only minimizes resource consumption during each server-client communication round but also safeguards client data privacy and ensures the preservation of the server's model privacy.

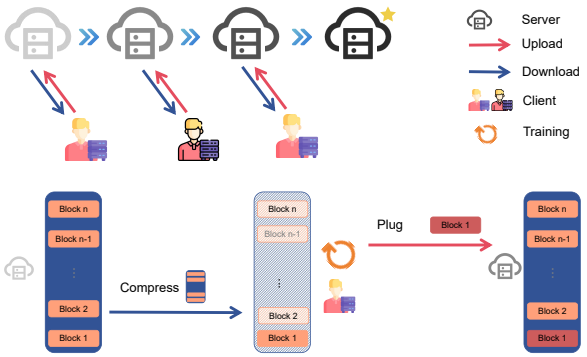


Figure 2: **Overview of FedCyBGD training.** In FedCyBGD, the server sends responsible (Block 1) and compressed unresponsible blocks (Others) to the client. The client fine-tunes the responsible block with local data, using the frozen compressed model. The refined block is returned to the server for integration. The process, involving only compressed parameters for download and block for upload, minimizes communication parameters. By updating only responsible blocks per client, computational and memory costs are reduced, enabling full-parameter tuning for FL on resource-limited edge devices.

Extensive experiments on various public LLMs and datasets, including GPT-2 (Radford et al., 2019), RoBERTa (Liu et al., 2019), ChatGLM-3 (Du et al., 2022), BLOOM (Le Scao et al., 2023), and LLaMA-2 (Touvron et al., 2023), underscore the resource efficiency and competitive performance of our FedCyBGD framework across a variety of NLP tasks, including text generation, classification, and question answering. Our thorough evaluation assesses the framework's resource costs in terms of communication, memory, and computation. In summary, FedCyBGD provides a practical, resource-effective full parameter tuning approach for FL.

The contributions of this study can be summarized as follows:

- **Cycle Block Update framework for full parameter tuning in FL**. Our framework, FedCyBGD, introduces a novel FL training approach that allows LLM training in the FL paradigm by cyclically updating the LLM blocks by clients. To the best of our knowledge, this is the first FL framework that enables LLM full parameter tuning on resource-limited edge devices, reducing computation, memory, and communication costs while maintaining comparable performance.

- **Compression for model download and block upload.** FedCyBGD incorporates designed compression schemes that enable the client to download only a parameter scaled-down proxy model, which reduces the client's download communication cost while keeping the responsible block properly updated.

- **Empirical Evaluation with resource Consumption Reduction.** Extensive experiments on various public LLMs across diverse datasets demonstrate FedCyBGD's significant performance in reducing resource usage (reduced to 13.8%) and improving training performance (up to 58.3% higher), while preserving model and data privacy.

2 Related Works

Federated Learning for LLMs. The training process of Large Language Models (LLMs) can generally be divided into two stages: pre-training and fine-tuning (Zhou et al., 2023; Touvron et al., 2023; Yang et al., 2024; Min et al., 2023). Pre-training typically involves adapting the LLM to large datasets, which usually requires comprehensive training of all parameters, a process also known as full fine-tuning. However, this approach can be prohibitively expensive, especially for federated learning where edge devices have limited capabilities (Imteaj et al., 2021; Ro et al., 2022).

Parameter Efficient Fine-Tuning (PEFT) methods such as Low-Rank Adaptation (LoRA) (Hu et al., 2021; Raje, 2024), Adapter (Houlsby et al., 2019), prompt- and prefix-tuning (Li and Liang, 2021; Lester et al., 2021), among others, are crucial for fine-tuning LLMs under memory resource constraints. The main idea of PEFT is to represent parameter updates in a much lower-dimensional subspace, significantly reducing memory consumption.

Despite the success of PEFT methods, fine-tuning within a substantially lower-dimensional subspace could potentially limit downstream performance (Zhang et al., 2024; Luo et al., 2024). Moreover, it requires knowledge of the entire model weights, which could compromise the privacy of data or model owners. Additionally, the fine-tuning process remains resource-intensive, as it necessitates at least one copy of the entire model to be placed on the device (Xiao et al., 2023a).

Several studies have built upon PEFT methods in the context of FL for LLM, including FedPETuning (Zhang et al., 2023), Federated Adapter Tuning (Cai et al., 2022), Federated Prompt Tuning (Zhao et al., 2023), and FedLora (Yi et al., 2023). However, these methods often sacrifice performance to reduce communication and computation overhead. While FL can benefit from PEFT methods, it still lacks a way to achieve full parameter tuning with an acceptable resource cost. In this work, we propose the cycle block gradient descent method, FedCyBGD, which makes it possible to save on communication, computation, and memory costs while adopting approximately full-parameter tuning. In addition, our framework is orthogonal to the PEFT methods, which can be utilized for further accelerating the tuning with reduced cost, as we shown in experiments of Section 4.

Compression for FL LLM communication. Model compression techniques aim to reduce the size of a machine learning model by altering its weights or structure (Zhu et al., 2023). Various compression methods have been developed to train computation-intensive fundamental models, including pruning (Han et al., 2015), quantization (Jacob et al., 2018; Xiao et al., 2023b), layer-drop (Sajjad et al., 2023), and knowledge distillation (Hinton et al., 2015), all of which aim to reduce the number of parameters transferred.

In the context of LLM training in FL, various compression algorithms have proven to be effective solutions for reducing communication, such as distilling knowledge between the server and clients (Tao et al., 2022; He et al., 2020; Fan et al., 2024). However, these methods often rely on the server's auxiliary dataset, which is typically scenario-constrained. Similarly, (Wu et al., 2024) compresses the communicated gradients by designing an autoencoder, while (Fang et al., 2024) designs an auto-low rank approximation method for the adapter. In our framework, we have minimized the main communication bottleneck, i.e., model uploading in FL (Raje, 2024). We aim to further reduce the downloading cost by avoiding the need to download the entire pre-trained model from the server. Instead, we propose using a smaller model that includes the necessary block. By utilizing this smaller model, we can update the corresponding block in the large model. To scale down the large model while maintaining its role in local updating, we apply model pruning (Ma et al., 2023) and layer drop (Xiao et al., 2023a) to the frozen blocks, reducing the downloading cost.

Algorithm 1 FedCyBGD

Input:Number of clients m, learning rate η_l and η_g, number of local epoch K, total training rounds T, client-block partition scheme π.
Output: Trained model parameter θ^T.
Initialize: Model parameter θ_0.

for $t = 0$ to $T-1$ **do**
  Assign each client i a responsible block based on partition scheme $\pi = \{\pi_1, \ldots, \pi_m\}$;
  for Client i in order **do**
    Send the compressed global model $\{\tilde{\theta}^{t+1}_{\pi_1}, \ldots, \tilde{\theta}^{t+1}_{\pi_{i-1}}, \theta^t_{\pi_i}, \tilde{\theta}^t_{\pi_{i+1}}, \ldots, \tilde{\theta}^t_{\pi_m}\}$ to client i, where $\tilde{\theta}$ is compressed model;
    for $k = 0$ to $K-1$ **do**
      $g^{t,k}_{\pi_i} \leftarrow \frac{\partial}{\partial \theta_{\pi_i}} \mathcal{L}(\tilde{\theta}^{t+1}_{\pi_1}, \ldots, \tilde{\theta}^{t+1}_{\pi_{i-1}}, \theta^{t,k-1}_{\pi_i}, \tilde{\theta}^t_{\pi_{i+1}}, \ldots, \tilde{\theta}^t_{\pi_m})$
    end for
    $\Delta^i_t = \theta^{t,K-1}_{\pi_i} - \theta^{t,0}_{\pi_i} = -\eta_L \sum_{k=0}^{K-1} g^{t,k}_{\pi_i}$;
    Upload updated block Δ^i_t;
  end for
  Server Update Block by $\theta^{t+1}_{\pi_i} = \theta^t_{\pi_i} + \eta \Delta^i_t$;
end for

3 The Proposed Method

In this section, we introduce FedCyBGD, a novel, simple, and effective training paradigm. It enables full-parameter tuning with resource costs acceptable for edge devices by allowing each client to cyclically participate in training one or more responsible blocks. We begin by providing an intriguing observation: cyclical updates are more beneficial for LLM tuning than the traditional FL paradigm with model aggregation. We then define the specific problem addressed in this study, followed by a detailed introduction to our approach.

3.1 Observation

Figure 1 presents intriguing observations for FL.

The first one is that the cyclical block update paradigm outperforms the traditional FL approach involving multiple clients and aggregation. Even with less computational resource cost, the cyclical update approach easily surpasses the performance of the aggregation paradigm. The first observation suggests that cyclical updates are an effective training paradigm for FL, outperforming the traditional aggregation strategy. This is because cyclical updates can potentially mitigate data heterogeneity and conflicting directions in aggregation. Aggregation forces different clients to focus on the same block update, amplifying the impact of heterogeneity. However, when clients update the entire model, their different datasets may lead to different update directions, reducing the likelihood of conflicts.

Another noteworthy observation is that block updates, in a cyclical manner, show superior performance compared to full parameter tuning in FL. The second observation indicates that the block update paradigm could be more effective than full model training. It offers more granular updates and potentially alleviates the problem of catastrophic forgetting caused by frequent updates. Similar findings are reported in Li et al. (2023), which demonstrates that personalizing each client's attention layer is more effective due to negative impacts of data heterogeneity in FL.

The observations inspire us to consider applying the cycle block model update as a more effective parameter tuning paradigm for LLM tuning in FL.

3.2 Problem Definition

In our Federated Learning (FL) framework, a single server manages a Large Language Model (LLM) $\mathcal{M}_\theta$, with parameters $\theta = \{\theta_1, \theta_2, \ldots, \theta_{|B|}\}$, where B denotes the model's blocks, such as the 12 transformer blocks in GPT-2-small, thus $|B| = 12$. There are m clients, each possessing a unique local dataset $\mathcal{D}_i$, with $\mathcal{D}_i \neq \mathcal{D}_j$ for $i \neq j$.

The goal of FL is to collaboratively pre-train or fine-tune the LLM using the clients' local datasets without data exchange, aiming to update the model from $\mathcal{M}_\theta$ to $\mathcal{M}_{\theta+\Delta}$, where $\Delta = \arg\min_\delta \mathcal{L}(\theta + \delta, \{\mathcal{D}_i\}_{i=1}^m)$. The objective is formulated as follows:

$$\begin{aligned}\theta^* &= \arg\min_\theta \mathcal{L}(\theta; \mathcal{D}) \\ &= \arg\min_\theta \frac{1}{m|\mathcal{D}_i|} \sum_{i\in[m]} \sum_{\xi_i \in \mathcal{D}_i} \mathcal{L}(\theta; \xi_i)\,, \end{aligned} \quad (1)$$

where $\xi_i \in \mathcal{D}_i$ is the local training data of client i.

However, from the above equation, we can conclude that it needs full parameter training, which is uncomfortable for clients as it consumes vast communication, computation, and memory. Fortunately, training one block θ_{π_i} for client i is more than acceptable, thus the objective can be approximated by:

$$\begin{aligned}\theta^* &= \{\theta^*_1, \ldots, \theta^*_{|B|}\} \\ &= \arg\min_{\theta_1,\ldots,\theta_{|B|}} \mathcal{L}(\theta_1, \ldots, \theta_{|B|}; \{\mathcal{D}_i\}_{i=1}^m)\,. \end{aligned} \quad (2)$$

Ideally, we separately update each block of the model, in order to finish the training or fine-tuning of the model within the resource-limited edge devices, and achieve similar performance compared

to directly optimizing the whole model $\mathcal{M}_\theta$ on overall data.

3.3 FedCyBGD

From above discussion, our primary objective is to reduce the resource cost while enabling full parameter LLM tuning for FL. We achieve this by proposing FedCyBGD, the cycle block update-based model tuning paradigm for FL, as illustrated in Figure 2 and describe the associated training algorithm in Algorithm 1.

Cycle Block Gradient Descent. A notable challenge that arises in objective 2 is to ensure that each block converges to the optimal model θ^*. Inspired by Adam Block Coordinate Descent (Luo et al., 2024), we can approximate the update for each block using the following problem:

$$\begin{aligned}
\theta_1^{t+1} &= \arg\min_{\theta_1} \mathcal{L}\left(\theta_1, \theta_2^t, \ldots, \theta_{|B|}^t\right) \\
\theta_2^{t+1} &= \arg\min_{\theta_2} \mathcal{L}\left(\theta_1^{t+1}, \theta_2, \ldots, \theta_{|B|}^t\right) \\
&\cdots \\
\theta_{|B|}^{t+1} &= \arg\min_{\theta_{|B|}} \mathcal{L}\left(\theta_1^{t+1}, \theta_2^{t+1}, \ldots, \theta_{|B|}\right)
\end{aligned}$$

Although the block update scheme allows for updating the large model progressively by using small blocks, it is a centralized learning paradigm that requires all data to participate in each block update. To address this challenge and leverage block coordinate descent for FL, we adopt the cycle update approach (Cho et al., 2023). This approach divides clients into different groups, and these groups take turns participating in FL training.

Based on our observation that aggregation is not beneficial for training fundamental models in FL, we treat each client as a block optimizer LLM tuning. Specifically, we select one client at a time to participate in training the responsible block, while keeping the other blocks fixed at their up-to-date values.

At the t-th block-epoch, updating the current active block of client i requires solving the following problem:

$$\begin{aligned}
&\theta_{\pi_i}^{t+1} \leftarrow \arg\min_{\theta_{\pi_i}} \mathbb{E}_{\xi_i \in \mathcal{D}_i} \\
&\quad \left\{\mathcal{L}\left(\theta_{\pi_1}^{t+1}, \ldots, \theta_{\pi_{i-1}}^{t+1}, \theta_{\pi_i}, \theta_{\pi_{i+1}}^{t}, \ldots, \theta_{\pi_m}^{t}; \xi_i\right)\right\}, \quad (3)
\end{aligned}$$

where π_i is the block partition scheme for client i, and θ_{π_i} represents the responsible block for client i. In this way, we utilize one client's local data to update a specific block of the LLM, alternating the parameter updates.

Since each client only trains and uploads one block, the memory, computation cost, and communication cost are significantly reduced. We will discuss the cost improvement in more detail later.

Compress LLM Downloading. The cycle block update method successfully reduces communication costs compared to traditional FL by having only one client train and upload the *responsible blocks* per cycle. However, it requires clients to download the entire model to update the responsible blocks, even though other *unresponsible blocks* remain frozen and aren't uploaded, leading to full model download requirements.

To tackle this challenge, we must devise a method to send only the responsible blocks to the client for updates, without transferring the entire model. This compels us to compress the model during transmission while preserving the integrity of responsible blocks. The other parts of the model, referred to as the emulator in (Xiao et al., 2023a), role in providing approximate gradient directions for updating the responsible blocks and closely resemble the frozen original components. The compression process with a randomized compression operator can be formulated as follows:

Definition 3.1. *A randomized map $\mathcal{C}: \mathbb{R}^d \mapsto \mathbb{R}^d$ is an ω-compression operator if*

$$\mathbb{E}[\mathcal{C}(\theta)] = \theta,\ \mathbb{E}\left[\|\mathcal{C}(\theta) - \theta\|^2\right] \leq \omega\|\theta\|^2,\ \forall \theta \in \mathbb{R}^d \quad (4)$$

In particular, no compression ($\mathcal{C}(\theta) \equiv \theta$) implies $\omega = 0$.

There are various ways to compress the original model $\{\theta_1, \ldots, \theta_i, \ldots, \theta_{|B|}\}$ into a low-parameter model, i.e., $\mathcal{C}(\theta)_i = \{\tilde{\theta}_{\pi_1}, \ldots, \theta_{\pi_i}, \tilde{\theta}_{\pi_m}\}$, where $\tilde{\theta}$ represented the compressed model parameter. Motivated by the experimental results from OffSite (Xiao et al., 2023a), we use a layer drop-based compression method that provides the best balance between the aforementioned criteria. Specifically, this method randomly drops out some layers from the frozen component, the the communicated model of client i can be formulated as:

$$\begin{aligned}
\mathcal{C}(\theta)_i = \{&\mathbb{I}_1\theta_1, \ldots, \mathbb{I}_{i-1}\theta_{i-1}, \\
&\theta_i, \mathbb{I}_{i+1}\theta_{i+1}, \ldots, \mathbb{I}_m\theta_m\}, \quad (5)
\end{aligned}$$

where $\mathbb{I}$ represents the indicator function, $\mathbb{I} = 0$ indicates that the layer should be dropped, which is controlled by a drop ratio.

Algorithm 2 Hybrid Pruning

Input: Server model θ, Training round T.
Output: Compressed Model $\tilde{\theta}$ to Client i.
Initialize: Round t, Updated Blocks Set $\mathbf{B}$, Non-updated Block Set $\overline{\mathbf{B}}$.
for block θ_j in $\theta = \{\theta_1, \ldots, \theta_m\}$ **do**
  if block has been updated, i.e., $\theta_j = \theta_j^{t+1}$ **then**
    $\mathbf{B} \leftarrow \mathbf{B} \cup \theta_j$
  else if $\theta_j = \theta_j^t$ **then**
    $\overline{\mathbf{B}} \leftarrow \overline{\mathbf{B}} \cup \theta_j$
  end if
end for
for $j \in [1, \ldots, m]$ **do**
  Compress: $\mathcal{C}(\theta)_j = \begin{cases} \mathbb{P}_j(\theta_j) & \text{if } j \in \mathbf{B}, \\ \mathbb{I}_j \theta_j & \text{if } j \in \overline{\mathbf{B}}. \end{cases}$
  $\tilde{\theta} \leftarrow \mathcal{C}(\theta)_j$
end for

Considering the cyclical update approach, with the current module as a reference point, the modules updated previously and those pending update exist in two distinct states, necessitating different compression strategies. For instance, if we uniformly treat all blocks, there's a risk of discarding all previously updated blocks in one round, rendering client i unable to benefit from prior model updates, thus leading to ineffective tuning. To address this, we devise a hybrid pruning method: we apply pruning to the updated blocks and layer dropping to the yet-to-be-updated blocks. Consequently, the compression operator for client i in round t is:

$$\mathcal{C}(\theta)_i^t = \{\mathbb{P}_1(\theta_1^{t+1}), \ldots, \mathbb{P}_{i-1}(\theta_{i-1}^{t+1}), \theta_i^t, \mathbb{I}_{i+1}\theta_{i+1}^t, \ldots, \mathbb{I}_m\theta_m^t\}, \quad (6)$$

where $\mathbb{P}$ represents a generic pruning function that can be dynamically chosen according to the task. In our paper, we follow the method described by Ma et al. (2023) for pruning the updated blocks. The compression process is described in Algorithm 2.

Connection to PEFT. Our framework is orthogonal to existing parameter-efficient fine-tuning methods for LLMs, such as LoRA (Hu et al., 2021) and Adaptor (Houlsby et al., 2019), as shown in Section 4. We focus on fully tuning the model with a cycle block update approach to reduce federated learning resource costs. Unlike PEFT, which uses low-rank parameter matrix approximation, our framework focuses on full parameter updates and can easily integrate PEFT by incorporating the parameter approximation into the updated blocks.

Computation, Memory, and Communication Complexity. FedCyBGD offers significant computational, memory, and communication efficiency by reducing the number of parameters updated and communicated during training. For example, when using FedCyBGD to fine-tune all parameters in LLaMA-2 7B compared to traditional full tuning in FL, it reduces 25% computation cost for a client for backward; each client requires about 17.3 GB, approximately 30% of the memory cost of traditional FL full parameter tuning; and it needs only 3.5 GB for parameter communication, reducing communication cost by roughly 75%. This results in faster training times and lower memory and communication requirements, making full parameter tuning of LLMs in an FL setting more feasible.

4 Numerical Results

In this section, we evaluate the proposed FedCyBGD on 5 datasets over 13 tasks in terms of memory consumption, running time, convergence, and downstream performance.

4.1 Experimental Setup

Models And Datasets. We evaluate FedCyBGD on large language models, including GPT2-small (Radford et al., 2019), BLOOM (Le Scao et al., 2023), RoBERTa-base (Liu et al., 2019), ChatGLM3-6B (Du et al., 2022) and LLaMA2-7B (Touvron et al., 2023). We evaluate language models on the GLUE benchmark (Wang et al., 2018), which is a benchmark including 10 widely used NLP datasets for evaluating the performance on a variety of tasks, including sentiment analysis, question answering, and textual entailment; and four other tasks including instruction following task on Alpaca-GPT4 (Peng et al., 2023), OpenAssistant Conversations task on OASST1 (Köpf et al., 2024), and QA task on SQuAD (Rajpurkar et al., 2016). For FL environment, we consider resource-limited 32 and 64 devices, and a parameter server. In each cycle, randomly choose a cyclic order for updating.

Baselines: We conduct a comparative analysis of our FedCyBGD framework against several baselines. These include LLM-ZS, which represents the zero-shot capabilities of LLM; Fed-full, a full parameter tuning method; two parameter-effective fine-tuning methods, namely Adaptor (Houlsby et al., 2019) and Lora (Hu et al.,

[1]To mitigate potential issues like Floating Point Exceptions or loss=NaN when using pure BF16 or FP16 for LoRA fine-tuning, we opted for BF16 mixed-precision training. As a result, FedLoRA still requires storing the model in Float32 format.

Method	Basic Model	Memory Cost	Download Cost	Upload Cost
Fed-full	12.61 GB	62.8 GB	12.48 GB	12.48 GB
FedLora	25.24 GB	27.4 GB	24.96 GB	32.03 MB
FedAdaptor	12.68 GB	15.8 GB	12.55 GB	62.93MB
FedCyBGD-Power	12.61 GB	16.4 GB	6.24 GB	770.37 MB
FedCyBGD	12.61 GB	17.2 GB	6.24 GB	770.37 MB

Table 1: Actual memory and communication costs of applying mixed precision training to finetune LLaMA-2 7B with gradient checkpointing. Note that Fed-full only supports FP16 precision training [1]. The maximum input sequence length is 4096 and the batch size is 2.

Methods	Forward	Backward	Update	Round Time
Fed-full	0.031 s	0.449 s	0.103 s	20030.8 s
FedLora	0.096 s	0.404 s	**0.001** s	34386.3 s
FedAdaptor	0.069 s	0.404 s	0.002 s	25462.0 s
FedCyBGD-Power	0.037 s	**0.302** s	0.003 s	18641.1 s
FedCyBGD	**0.030** s	0.338 s	0.102 s	**18359.1** s

Table 2: The time spent per epoch on forward, backward, and update processes for fine-tuning the LLaMA 2-7B model on Alpaca-GPT4 is detailed. The term 'round time' refers to the time taken per global round of fine-tuning, which is completed when all clients finish their participation. FedCyBGD-Power refers to using the PowerSGD as an optimizer instead of using the ADAM optimizer. The best results are highlighted in **bold** and the second bests are marked with underline.

2021); BADAM (Luo et al., 2024), another centralized effective LLM training method; and two optimization methods, ADAM (Kingma and Ba, 2014) and PowerSGD (Vogels et al., 2019). Unless otherwise specified, we use the ADAM optimizer as the backbone optimization method. For a fair comparison, we adapt the centralized PEFT methods, specifically Adapter and Lora, to the federated learning setting. More details about the experiment setup are provided in Appendix 7.

4.2 Evaluation of FedCyBGD

Memory and Communication Consumption. Table 1 demonstrates that FedCyBGD is effective in reducing the communicated parameter and memory costs compared to existing full-parameter tuning paradigms or the PEFT training method. Specifically, it reduces memory consumption by approximately 72.7% compared to existing full-parameter tuning methods. In terms of communication, even when compared to the best-performing PEFT method, our full-parameter tuning framework reduces communication costs by about 44.1%.

Wall-clock running time comparison. The time consumption for each method is divided into forward, backward, and update phases, as detailed in Table 2. For the LLaMA2 model fine-tuning, the forward times of Fed-full and FedCyBGD are similar, as both process the entire model. FedLora and FedAdaptor incur slightly more time due to extra steps in activation registration and low-rank adapter computations, respectively. Notably, FedCyBGD cuts the backward time nearly in half compared to Fed-full and FedLora.

Performance improvement by FedCyBGD. Table 3 illustrates that our method achieves competitive performance with traditional full parameter tuning while significantly reducing memory and communication costs (reduced to 13.8%). Furthermore, we demonstrate the consistent effectiveness of FedCyBGD by evaluating it on various LLMs and datasets. Specifically, in Table 4, FedCyBGD consistently outperforms baselines in two different datasets, Alpaca-GPT4 and OASST1, using large-sized models LLaMA2-7B and ChatGLM-6B, and achieves competitive performance with full tuning in Bloom-1.3B. Additionally, in the question-answering task, FedCyBGD exhibits a significant improvement over FedAdaptor, with up to 58.3% higher performance and minimal communication cost.

Orthogonal to PEFT. Table 6 demonstrates that FedCyBGD is compatible with existing PEFT algorithms such as LoRA and Adaptor. By utilizing PEFT, FedCyBGD successfully reduces communication and memory costs, albeit with a slight sacrifice in performance. Furthermore, the table highlights the effectiveness of our compression method, which significantly reduces communication while maintaining a relatively small impact on performance.

Additional Ablation Studies. To thoroughly validate the effectiveness of our method, we conducted experiments under various settings, which included different numbers of clients, diverse batch sizes, and block allocation strategies for clients, as de-

Method	Memory	Communication	CoLA	MRPC	MNLI	MNLI-mm	QNLI	QQP	RTE	SST-2	stsb
Fed-full	15.87GB	915.52MB	0.57	0.87	0.88	0.88	0.93	0.91	0.73	0.94	0.90
FedLora	13.67GB	463.36MB	0.45	0.72	0.54	0.55	0.68	0.75	0.55	0.82	0.59
FedAdaptor	13.12GB	466.97MB	0.42	0.71	0.54	0.55	0.71	0.76	0.56	0.82	0.43
FedCyBGD	2.79GB	591.93MB	0.52	0.80	0.87	0.87	0.92	0.90	0.57	0.93	0.88

Table 3: Evaluating FedCyBGD for full parameter tuning on GLUE benchmark using pre-trained RoBERTa-base. Higher scores on these tasks represent better performance.

Task	Method	ChatGLM-6B	Bloom-1.3B	LLaMA2-7B
Alpaca-GPT4 ↓	Fed-full	1.27	**1.47**	0.92
	BADAM	1.22	1.57	0.90
	FedLora	-	1.64	0.90
	FedAdaptor	1.26	1.57	0.90
	FedCyBGD-Power	1.26	1.93	0.93
	FedCyBGD	**1.21**	1.56	**0.90**
OASST1 ↓	Fed-full	1.85	**1.85**	1.14
	BADAM	1.69	1.88	1.14
	FedLora	-	1.92	1.14
	FedAdaptor	1.72	1.90	1.12
	FedCyBGD-Power	2.13	2.09	1.13
	FedCyBGD	**1.62**	1.86	**1.12**

Table 4: Performance Comparison of FedCyBGD and baseline approaches. We assessed FedCyBGD's generalization across three models and two tasks. Identical values with different ranks (**bold** for best, underline for second-best) indicate unseen decimal differences.

	FedFT	BADAM	FedAdapter	FedCyBGD
Exact Match ↑	**68.14**	47.32	9.07	67.36
F1 ↑	**79.03**	60.29	16.28	77.82
Memory	16.80GB	15.21GB	13.65GB	15.21GB
Communication	974MB	514MB	493.8MB	514MB

Table 5: Comparison of FedCyBGD and baselines on SQuAD QA task. All results are obtained using GPT-2 small, batch size 24. Exact Match and F1 are two evaluation metrics for QA task

Method	Performance ↓	Memory Cost	Communication Cost
FedCyBGD	0.9	17.20GB	13.23GB
+ i	0.91	17.20GB	6.99GB
+ ii	1.04	26.89GB	12.49GB
+ iii	1.11	14.20GB	6.30GB

Table 6: The performance of FedCyBGD, compatible with compression and PEFT methods, on Alpaca-GPT4 dataset with LLaMa-2. Here, FedCyBGD refers to the cyclic block update without compressed download. i) represents the hybrid compression method; ii) represents the LoRA method with rank 16; and iii) refers to the Adaptor tuning method.

tailed in Appendix 7. It is noteworthy that: 1) our method demonstrates increasingly apparent memory efficiency advantages over the PEFT method as the batch size grows; and 2) our method exhibits robustness to variations in both the block allocation strategy and the number of clients.

5 Conclusions

In this work, we introduce FedCyBGD, a novel, simple, and effective full parameter tuning framework for federated Large Language Models (LLMs). This pioneering approach allows users to cyclically update blocks, thereby facilitating the tuning of federated LLMs. FedCyBGD significantly reduces computation, memory, and communication costs for resource-limited edge clients in Federated Learning (FL), enabling full parameter training and tuning of LLaMA-2 7B using a single RTX 3090-24GB GPU. To the best of our knowledge, FedCyBGD is the first framework that enables full parameter LLM pre-training or fine-tuning for federated learning settings on a single consumer-level GPU. Extensive empirical results on various LLMs and NLP tasks demonstrate the framework's effectiveness in terms of accuracy (up to 58.3% higher) and efficiency (reduced to 13.8%).

6 Limitations

While our method can significantly reduce the resource cost for federated learning to achieve full parameter tuning LLMs, we still lack theoretical guarantees regarding the convergence of the cycle block update. Previous work on cycle updates analysis, such as (Cho et al., 2023), can be utilized in the block update paradigm. Furthermore, our focus has primarily been on applying FedCyBGD to NLP tasks, but extending its application to other tasks, such as visual tasks, presents an opportunity to showcase the capabilities of FedCyBGD. We leave these directions for future improvements.

References

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Dongqi Cai, Yaozong Wu, Shangguang Wang, Felix Xiaozhu Lin, and Mengwei Xu. 2022. Fedadapter: Efficient federated learning for modern nlp. *arXiv preprint arXiv:2205.10162*.

Tianshi Che, Ji Liu, Yang Zhou, Jiaxiang Ren, Jiwen Zhou, Victor S Sheng, Huaiyu Dai, and Dejing Dou. 2023. Federated learning of large language models with parameter-efficient prompt tuning and adaptive optimization. *arXiv preprint arXiv:2310.15080*.

Yae Jee Cho, Pranay Sharma, Gauri Joshi, Zheng Xu, Satyen Kale, and Tong Zhang. 2023. On the convergence of federated averaging with cyclic client participation. In *International Conference on Machine Learning*, pages 5677–5721. PMLR.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.

Zhengxiao Du, Yujie Qian, Xiao Liu, Ming Ding, Jiezhong Qiu, Zhilin Yang, and Jie Tang. 2022. Glm: General language model pretraining with autoregressive blank infilling. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 320–335.

Tao Fan, Yan Kang, Guoqiang Ma, Weijing Chen, Wenbin Wei, Lixin Fan, and Qiang Yang. 2023. Fate-llm: A industrial grade federated learning framework for large language models. *arXiv preprint arXiv:2310.10049*.

Tao Fan, Guoqiang Ma, Yan Kang, Hanlin Gu, Lixin Fan, and Qiang Yang. 2024. Fedmkt: Federated mutual knowledge transfer for large and small language models. *arXiv preprint arXiv:2406.02224*.

Yuxin Fang, Wen Wang, Binhui Xie, Quan Sun, Ledell Wu, Xinggang Wang, Tiejun Huang, Xinlong Wang, and Yue Cao. 2023. Eva: Exploring the limits of masked visual representation learning at scale. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19358–19369.

Zihan Fang, Zheng Lin, Zhe Chen, Xianhao Chen, Yue Gao, and Yuguang Fang. 2024. Automated federated pipeline for parameter-efficient fine-tuning of large language models. *arXiv preprint arXiv:2404.06448*.

Song Han, Huizi Mao, and William J Dally. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*.

Chaoyang He, Murali Annavaram, and Salman Avestimehr. 2020. Group knowledge transfer: Federated learning of large cnns at the edge. *Advances in Neural Information Processing Systems*, 33:14068–14080.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for nlp. In *International conference on machine learning*, pages 2790–2799. PMLR.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Ahmed Imteaj, Urmish Thakker, Shiqiang Wang, Jian Li, and M Hadi Amini. 2021. A survey on federated learning for resource-constrained iot devices. *IEEE Internet of Things Journal*, 9(1):1–24.

Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2704–2713.

Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. 2019. Advances and

open problems in federated learning. *arXiv preprint arXiv:1912.04977*.

Georgios A Kaissis, Marcus R Makowski, Daniel Rückert, and Rickmer F Braren. 2020. Secure, privacy-preserving and federated machine learning in medical imaging. *Nature Machine Intelligence*, 2(6):305–311.

Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

Andreas Köpf, Yannic Kilcher, Dimitri von Rütte, Sotiris Anagnostidis, Zhi Rui Tam, Keith Stevens, Abdullah Barhoum, Duc Nguyen, Oliver Stanley, Richárd Nagyfi, et al. 2024. Openassistant conversations-democratizing large language model alignment. *Advances in Neural Information Processing Systems*, 36.

Weirui Kuang, Bingchen Qian, Zitao Li, Daoyuan Chen, Dawei Gao, Xuchen Pan, Yuexiang Xie, Yaliang Li, Bolin Ding, and Jingren Zhou. 2023. Federatedscope-llm: A comprehensive package for fine-tuning large language models in federated learning. *arXiv preprint arXiv:2309.00363*.

Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. 2023. Bloom: A 176b-parameter open-access multilingual language model.

Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*.

Hongxia Li, Zhongyi Cai, Jingya Wang, Jiangnan Tang, Weiping Ding, Chin-Teng Lin, and Ye Shi. 2023. Fedtp: Federated learning by transformer personalization. *IEEE transactions on neural networks and learning systems*.

Qinbin Li, Zeyi Wen, Zhaomin Wu, Sixu Hu, Naibo Wang, Yuan Li, Xu Liu, and Bingsheng He. 2021. A survey on federated learning systems: Vision, hype and reality for data privacy and protection. *IEEE Transactions on Knowledge and Data Engineering*, 35(4):3347–3366.

Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.

Qijun Luo, Hengxu Yu, and Xiao Li. 2024. Badam: A memory efficient full parameter training method for large language models. *arXiv preprint arXiv:2404.02827*.

Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems*, 36:21702–21720.

Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR.

AI Meta. 2024. Introducing meta llama 3: The most capable openly available llm to date. *Meta AI*.

Bonan Min, Hayley Ross, Elior Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. 2023. Recent advances in natural language processing via large pre-trained language models: A survey. *ACM Computing Surveys*, 56(2):1–40.

Viraaji Mothukuri, Reza M Parizi, Seyedamin Pouriyeh, Yan Huang, Ali Dehghantanha, and Gautam Srivastava. 2021. A survey on security and privacy of federated learning. *Future Generation Computer Systems*, 115:619–640.

Baolin Peng, Chunyuan Li, Pengcheng He, Michel Galley, and Jianfeng Gao. 2023. Instruction tuning with gpt-4. *arXiv preprint arXiv:2304.03277*.

Youyang Qu, Shiva Raj Pokhrel, Sahil Garg, Longxiang Gao, and Yong Xiang. 2020. A blockchained federated learning framework for cognitive computing in industry 4.0 networks. *IEEE Transactions on Industrial Informatics*, 17(4):2964–2973.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR.

Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning*, pages 28492–28518. PMLR.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.

Arian Raje. 2024. *Communication-Efficient LLM Training for Federated Learning*. Ph.D. thesis, Carnegie Mellon University Pittsburgh, PA.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250*.

Jae Hun Ro, Theresa Breiner, Lara McConnaughey, Mingqing Chen, Ananda Theertha Suresh, Shankar Kumar, and Rajiv Mathews. 2022. Scaling language model size in cross-device federated learning. *arXiv preprint arXiv:2204.09715*.

Hassan Sajjad, Fahim Dalvi, Nadir Durrani, and Preslav Nakov. 2023. On the effect of dropping layers of pre-trained transformer models. *Computer Speech & Language*, 77:101429.

Yuris Mulya Saputra, Dinh Thai Hoang, Diep N Nguyen, Eryk Dutkiewicz, Markus Dominik Mueck, and Srikathyayani Srikanteswara. 2019. Energy demand prediction with federated learning for electric vehicle networks. In *2019 IEEE global communications conference (GLOBECOM)*, pages 1–6. IEEE.

Jiang Tao, Zhen Gao, and Zhaohui Guo. 2022. Training vision transformers in federated learning with limited edge-device resources. *Electronics*, 11(17):2638.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Thijs Vogels, Sai Praneeth Karimireddy, and Martin Jaggi. 2019. Powersgd: Practical low-rank gradient compression for distributed optimization. *Advances in Neural Information Processing Systems*, 32.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. 2018. Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*.

Haozhao Wang, Zhihao Qu, Qihua Zhou, Haobo Zhang, Boyuan Luo, Wenchao Xu, Song Guo, and Ruixuan Li. 2021. A comprehensive survey on training acceleration for large machine learning models in iot. *IEEE Internet of Things Journal*, 9(2):939–963.

Huiwen Wu, Xiaohan Li, Deyi Zhang, Xiaogang Xu, Jiafei Wu, Puning Zhao, and Zhe Liu. 2024. Cg-fedllm: How to compress gradients in federated funetuning for large language models. *arXiv preprint arXiv:2405.13746*.

Guangxuan Xiao, Ji Lin, and Song Han. 2023a. Offsite-tuning: Transfer learning without full model. *arXiv preprint arXiv:2302.04870*.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023b. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099. PMLR.

Jingfeng Yang, Hongye Jin, Ruixiang Tang, Xiaotian Han, Qizhang Feng, Haoming Jiang, Shaochen Zhong, Bing Yin, and Xia Hu. 2024. Harnessing the power of llms in practice: A survey on chatgpt and beyond. *ACM Transactions on Knowledge Discovery from Data*, 18(6):1–32.

Qiang Yang, Lixin Fan, and Han Yu. 2020. *Federated learning: Privacy and incentive*, volume 12500. Springer Nature.

Rui Ye, Wenhao Wang, Jingyi Chai, Dihan Li, Zexi Li, Yinda Xu, Yaxin Du, Yanfeng Wang, and Siheng Chen. 2024. Openfedllm: Training large language models on decentralized private data via federated learning. *arXiv preprint arXiv:2402.06954*.

Liping Yi, Han Yu, Gang Wang, and Xiaoguang Liu. 2023. Fedlora: Model-heterogeneous personalized federated learning with lora tuning. *arXiv preprint arXiv:2310.13283*.

Biao Zhang, Zhongtao Liu, Colin Cherry, and Orhan Firat. 2024. When scaling meets llm finetuning: The effect of data, model and finetuning method. *arXiv preprint arXiv:2402.17193*.

Zhuo Zhang, Yuanhang Yang, Yong Dai, Qifan Wang, Yue Yu, Lizhen Qu, and Zenglin Xu. 2023. Fedpetuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models. In *Annual Meeting of the Association of Computational Linguistics 2023*, pages 9963–9977. Association for Computational Linguistics (ACL).

Haodong Zhao, Wei Du, Fangqi Li, Peixuan Li, and Gongshen Liu. 2023. Fedprompt: Communication-efficient and privacy-preserving prompt tuning in federated learning. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE.

Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. 2024. Galore: Memory-efficient llm training by gradient low-rank projection. *arXiv preprint arXiv:2403.03507*.

Ce Zhou, Qian Li, Chen Li, Jun Yu, Yixin Liu, Guangjing Wang, Kai Zhang, Cheng Ji, Qiben Yan, Lifang He, et al. 2023. A comprehensive survey on pretrained foundation models: A history from bert to chatgpt. *arXiv preprint arXiv:2302.09419*.

Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. 2023. A survey on model compression for large language models. *arXiv preprint arXiv:2308.07633*.

Shengyao Zhuang, Bing Liu, Bevan Koopman, and Guido Zuccon. 2023a. Open-source large language models are strong zero-shot query likelihood models for document ranking. *arXiv preprint arXiv:2310.13243*.

Yan Zhuang, Qi Liu, Yuting Ning, Weizhe Huang, Rui Lv, Zhenya Huang, Guanhao Zhao, Zheng Zhang, Qingyang Mao, Shijin Wang, et al. 2023b. Efficiently measuring the cognitive ability of llms:

An adaptive testing perspective. *arXiv preprint arXiv:2306.10512*.

7 Experiment Details

7.1 Experimental Environment

For all experiments, we use NVIDIA GeForce RTX 3090 and H20 GPUs.

7.2 Experimental Setup

General setup. In this paper, we present the evaluation of FedCyBGD on several large language models, including GPT2-small (Radford et al., 2019), BLOOM (Le Scao et al., 2023), RoBERTa-base (Liu et al., 2019), ChatGLM3-6B (Du et al., 2022), and LLaMA2-7B (Touvron et al., 2023). The evaluation is conducted using the GLUE benchmark (Wang et al., 2018), which comprises 10 widely used NLP datasets designed to assess performance across a variety of tasks such as sentiment analysis, question answering, and textual entailment. Additionally, we include four other tasks: the instruction following task on Alpaca-GPT4 (Peng et al., 2023), the OpenAssistant Conversations task on OASST1 (Köpf et al., 2024), and the QA task on SQuAD (Rajpurkar et al., 2016).

For the federated learning (FL) environment, we consider scenarios involving resource-limited devices, specifically configurations with 32 and 64 devices, along with a parameter server. In each training cycle, a random cyclic order is chosen for updating the models. This setup aims to simulate realistic conditions and evaluate the robustness and efficiency of FedCyBGD under varying resource constraints.

In this section, we conduct a comparative analysis of our FedCyBGD framework against several baseline methods. The baselines include LLM-ZS, which represents the zero-shot capabilities of large language models (LLMs); Fed-full, a full parameter tuning method; and two parameter-efficient fine-tuning methods, namely Adaptor (Houlsby et al., 2019) and Lora (Hu et al., 2021). Additionally, we compare against BADAM (Luo et al., 2024), a centralized effective LLM training method, and two optimization methods, ADAM (Kingma and Ba, 2014) and PowerSGD (Vogels et al., 2019). Unless otherwise specified, the ADAM optimizer is used as the backbone optimization method.

7.3 Additional results

Method	ChatGLM	BLOOM	LLaMA-2
Fed-full	47341.2 s	29074.8 s	20030.8 s
FedLora	-	22815.0 s	34386.3 s
FedAdaptor	-	21261.2 s	25462.0 s
FedCyBGD-Power	41374.21 s	19845.7 s	18641.1 s
FedCyBGD	40241.45 s	18092.7s	18359.1 s

Table 7: Time spent per federated client participation round for fine-tuning LLaMA2-7B on Alpaca-GPT4 with batch size 2.

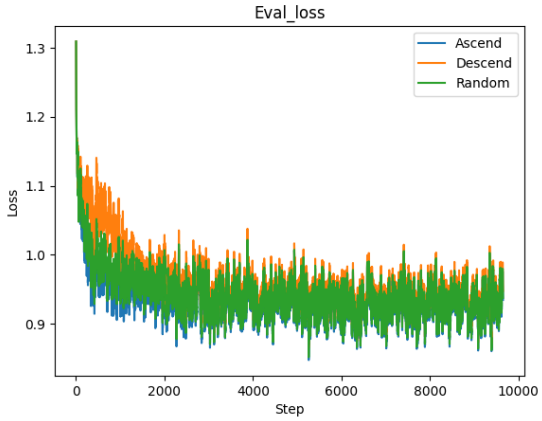


Figure 3: The performance of FedCyBGD under different block allocation strategies.